

EECS 348 Assignment 2

a & b. GenAI Programs Used:

Two GenAI programs were used to generate an initial C implementation: GPT-5.6 Luna (accessed via the ChatGPT web interface at chatgpt.com) and Gemini 3.1 Pro (accessed via the Gemini web interface at gemini.google.com).

c. Prompt Used:

"Write a C program that implements a MaxHeap-based priority queue using a list (array) implementation, built entirely from scratch with no pre-existing heap libraries. The program prioritizes a CEO's emails. Read commands from standard input: EMAIL <sender category>,<subject>,<date> (categories: Boss, Subordinate, Peer, ImportantPerson, OtherPerson, priority in that order, Boss highest), NEXT (show but don't remove the top email: Sender/Subject/Date), READ (remove the top email, no output), COUNT (print "There are X emails to read."). Within the same category, the newest date (MM-DD-YYYY) goes first. Handle edge cases: NEXT/READ on an empty queue, back-to-back NEXTs, back-to-back READs."

d. Generated Code:

ChatGPT:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_CAPACITY 10

// Represents one email
typedef struct {
    char sender[50];
    char subject[200];
    char date[11];    // MM-DD-YYYY
    int priority;     // Higher number = higher priority
} Email;

// Represents the MaxHeap priority queue
typedef struct {
    Email *emails;
    int size;
    int capacity;
} MaxHeap;

// Return the priority of a sender category
int getPriority(char *sender)
```

```

{
    if (strcmp(sender, "Boss") == 0)
        return 5;
    else if (strcmp(sender, "Subordinate") == 0)
        return 4;
    else if (strcmp(sender, "Peer") == 0)
        return 3;
    else if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    else
        return 1; // OtherPerson
}

```

```

// Convert MM-DD-YYYY into a number that can be compared
// YYYY is most significant, then MM, then DD.
int dateValue(char *date)
{
    int month, day, year;

    sscanf(date, "%d-%d-%d", &month, &day, &year);

    return year * 10000 + month * 100 + day;
}

```

```

// Return 1 if email1 should come before email2
int higherPriority(Email *email1, Email *email2)
{
    // Compare sender categories first
    if (email1->priority > email2->priority)
        return 1;

    if (email1->priority < email2->priority)
        return 0;

    // Same category: newer date comes first
    if (dateValue(email1->date) > dateValue(email2->date))
        return 1;

    return 0;
}

```

```
// Swap two emails
void swap(Email *a, Email *b)
{
    Email temp = *a;
    *a = *b;
    *b = temp;
}
```

```
// Move an email upward in the heap
void heapifyUp(MaxHeap *heap, int index)
{
    int parent;

    while (index > 0)
    {
        parent = (index - 1) / 2;

        if (higherPriority(&heap->emails[index],
                        &heap->emails[parent]))
        {
            swap(&heap->emails[index], &heap->emails[parent]);
            index = parent;
        }
        else
        {
            break;
        }
    }
}
```

```
// Move an email downward in the heap
void heapifyDown(MaxHeap *heap, int index)
{
    int left;
    int right;
    int largest;

    while (1)
    {
        left = 2 * index + 1;
        right = 2 * index + 2;
        largest = index;
```

```

// Check left child
if (left < heap->size &&
    higherPriority(&heap->emails[left],
                  &heap->emails[largest]))
{
    largest = left;
}

// Check right child
if (right < heap->size &&
    higherPriority(&heap->emails[right],
                  &heap->emails[largest]))
{
    largest = right;
}

// No child has higher priority
if (largest == index)
    break;

swap(&heap->emails[index], &heap->emails[largest]);

index = largest;
}
}

```

```

// Add an email to the MaxHeap
void insert(MaxHeap *heap, Email email)
{
    // Increase array size if necessary
    if (heap->size == heap->capacity)
    {
        heap->capacity *= 2;

        heap->emails = realloc(
            heap->emails,
            heap->capacity * sizeof(Email)
        );
    }

    // Put new email at the end
    heap->emails[heap->size] = email;
}

```

```

// Move it to the correct position
heapifyUp(heap, heap->size);

heap->size++;
}

// Remove the highest-priority email
void removeMax(MaxHeap *heap)
{
    // Nothing to remove
    if (heap->size == 0)
        return;

    // Move the last element to the root
    heap->emails[0] = heap->emails[heap->size - 1];

    heap->size--;

    // Restore MaxHeap property
    if (heap->size > 0)
        heapifyDown(heap, 0);
}

// Print the highest-priority email without removing it
void nextEmail(MaxHeap *heap)
{
    if (heap->size == 0)
    {
        return;
    }

    printf("%s/%s/%s\n",
        heap->emails[0].sender,
        heap->emails[0].subject,
        heap->emails[0].date);
}

// Print number of emails
void countEmails(MaxHeap *heap)
{

```

```
    printf("There are %d emails to read.\n", heap->size);  
}
```

```
int main(void)  
{  
    MaxHeap heap;  
  
    heap.size = 0;  
    heap.capacity = INITIAL_CAPACITY;  
  
    heap.emails = malloc(heap.capacity * sizeof(Email));  
  
    char line[300];  
  
    // Read commands until EOF  
    while (fgets(line, sizeof(line), stdin) != NULL)  
    {  
        // Remove newline  
        line[strcspn(line, "\n")] = '\0';  
  
        // EMAIL command  
        if (strncmp(line, "EMAIL ", 6) == 0)  
        {  
            Email email;  
  
            // Skip "EMAIL "  
            char *data = line + 6;  
  
            // Split input at commas  
            char *sender = strtok(data, ",");  
            char *subject = strtok(NULL, ",");  
            char *date = strtok(NULL, ",");  
  
            if (sender != NULL && subject != NULL && date != NULL)  
            {  
                strcpy(email.sender, sender);  
                strcpy(email.subject, subject);  
                strcpy(email.date, date);  
  
                email.priority = getPriority(email.sender);  
  
                insert(&heap, email);  
            }  
        }  
    }  
}
```

```

    }

    // NEXT command
    else if (strcmp(line, "NEXT") == 0)
    {
        nextEmail(&heap);
    }

    // READ command
    else if (strcmp(line, "READ") == 0)
    {
        removeMax(&heap);
    }

    // COUNT command
    else if (strcmp(line, "COUNT") == 0)
    {
        countEmails(&heap);
    }
}

free(heap.emails);

return 0;
}

```

Gemini:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 10000
#define MAX_LINE 256

/* Enum to easily compare sender category priorities */
typedef enum {
    OTHER_PERSON = 0,
    IMPORTANT_PERSON = 1,
    PEER = 2,
    SUBORDINATE = 3,
    BOSS = 4
} PriorityLevel;

```

```

typedef struct {
    char category[32];
    char subject[128];
    char date_str[16];
    PriorityLevel priority;
    int year;
    int month;
    int day;
} Email;

Email heap[MAX_EMAILS];
int heap_size = 0;

/* Maps a category string to its priority enum */
PriorityLevel get_priority(const char* category) {
    if (strcmp(category, "Boss") == 0) return BOSS;
    if (strcmp(category, "Subordinate") == 0) return SUBORDINATE;
    if (strcmp(category, "Peer") == 0) return PEER;
    if (strcmp(category, "ImportantPerson") == 0) return IMPORTANT_PERSON;
    return OTHER_PERSON;
}

/*
 * Returns > 0 if a has higher priority than b,
 *      < 0 if b has higher priority than a,
 *      0 if equal.
 */
int compare_emails(const Email *a, const Email *b) {
    if (a->priority != b->priority) {
        return a->priority - b->priority;
    }
    /* Tie-breaker: Date (Newest first) */
    if (a->year != b->year) return a->year - b->year;
    if (a->month != b->month) return a->month - b->month;
    return a->day - b->day;
}

void swap(Email *a, Email *b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

void heapify_up(int index) {

```

```

    int parent = (index - 1) / 2;
    while (index > 0 && compare_emails(&heap[index], &heap[parent]) > 0) {
        swap(&heap[index], &heap[parent]);
        index = parent;
        parent = (index - 1) / 2;
    }
}

```

```

void heapify_down(int index) {
    int largest = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left < heap_size && compare_emails(&heap[left], &heap[largest]) > 0) {
        largest = left;
    }
    if (right < heap_size && compare_emails(&heap[right], &heap[largest]) > 0) {
        largest = right;
    }

    if (largest != index) {
        swap(&heap[index], &heap[largest]);
        heapify_down(largest);
    }
}

```

```

void insert_email(const char *category, const char *subject, const char *date_str) {
    if (heap_size >= MAX_EMAILS) return; /* Heap is full */

    Email new_email;
    strncpy(new_email.category, category, sizeof(new_email.category) - 1);
    new_email.category[sizeof(new_email.category) - 1] = '\0';

    strncpy(new_email.subject, subject, sizeof(new_email.subject) - 1);
    new_email.subject[sizeof(new_email.subject) - 1] = '\0';

    strncpy(new_email.date_str, date_str, sizeof(new_email.date_str) - 1);
    new_email.date_str[sizeof(new_email.date_str) - 1] = '\0';

    new_email.priority = get_priority(category);
    sscanf(date_str, "%d-%d-%d", &new_email.month, &new_email.day, &new_email.year);

    heap[heap_size] = new_email;
    heapify_up(heap_size);
}

```

```

    heap_size++;
}

void peek_email() {
    if (heap_size == 0) {
        printf("Queue is empty.\n");
        return;
    }
    printf("%s/%s/%s\n", heap[0].category, heap[0].subject, heap[0].date_str);
}

void read_email() {
    if (heap_size == 0) {
        return; /* Do nothing on empty queue as per output silence requirement */
    }
    heap[0] = heap[heap_size - 1];
    heap_size--;
    if (heap_size > 0) {
        heapify_down(0);
    }
}

void print_count() {
    printf("There are %d emails to read.\n", heap_size);
}

/* Helper to strip trailing newlines */
void strip_newline(char *str) {
    size_t len = strlen(str);
    while (len > 0 && (str[len - 1] == '\n' || str[len - 1] == '\r')) {
        str[len - 1] = '\0';
        len--;
    }
}

int main() {
    char line[MAX_LINE];

    while (fgets(line, sizeof(line), stdin)) {
        strip_newline(line);
        if (strlen(line) == 0) continue;

        if (strncmp(line, "EMAIL ", 6) == 0) {
            char *payload = line + 6;

```

```

char *first_comma = strchr(payload, ',');
char *last_comma = strrchr(payload, ',');

if (first_comma && last_comma && first_comma != last_comma) {
    *first_comma = '\0';
    *last_comma = '\0';

    char *category = payload;
    char *subject = first_comma + 1;
    char *date_str = last_comma + 1;

    insert_email(category, subject, date_str);
}
}
else if (strcmp(line, "NEXT") == 0) {
    peek_email();
}
else if (strcmp(line, "READ") == 0) {
    read_email();
}
else if (strcmp(line, "COUNT") == 0) {
    print_count();
}
}

return 0;
}

```

e. Code Analysis

Both GPT-5.6 Luna and Gemini 3.1 Pro produced list-based (array-backed) MaxHeap implementations with correct heap logic: emails are ranked by sender category (Boss > Subordinate > Peer > ImportantPerson > OtherPerson) with ties broken by newest date first, and both correctly implement heapify-up on insert and heapify-down on removal. Testing both against the assignment's sample test file confirmed the heap ordering itself is correct in both cases: the top email after all six were queued was Boss / "Never Mind" / 01-03-2025 as expected, and the unread count updated correctly after each READ.

However, both implementations share the same correctness flaw: the NEXT command prints Sender/Subject/Date on a single line separated by slashes, rather than the required multi-line format ("Next email:" followed by separate Sender:, Subject:, and Date: lines). Since the assignment is graded against a specific output format, both raw outputs would fail an exact-match grading script despite having correct underlying heap logic. Gemini 3.1 Pro additionally prints "Queue is empty." when NEXT is called on an empty heap, an extra line the

spec never asks for, which could cause a mismatch against expected output on edge-case tests.

Time complexity: both use a proper array-based heap with $O(\log n)$ insert and remove operations; neither cheats by using a sorted list or linear scan. Gemini 3.1 Pro's `heapify_down` is written recursively rather than iteratively, which adds a small, likely negligible amount of call-stack overhead at this scale.

Space complexity: GPT-5.6 Luna uses a dynamically growable array that starts small and doubles its capacity via `realloc` as needed. Gemini 3.1 Pro instead allocates a fixed global array of 10,000 emails up front. This makes Gemini's version less memory-efficient for small inboxes and, more importantly, silently drops any email past the 10,000-email cap rather than growing to accommodate it.

Maintainability: both are readable with function-level comments, but neither meets the assignment's comment rubric: no prolog comment block (program name, description, inputs, outputs, author, dates), no per-line or per-block explanation of what each line does, and no attribution of collaborators or code sources.

f. Selection and Justification

GPT-5.6 Luna's version was selected as the baseline. While Gemini 3.1 Pro's data modeling was slightly cleaner (an enum for priority levels and separate year/month/day integers instead of GPT-5.6 Luna's single YYYYMMDD integer trick), Gemini's fixed-capacity array is a real scalability weakness for a program meant to model a busy CEO's growing inbox, and its extra "Queue is empty." output is a correctness risk against a script-based grader. GPT-5.6 Luna's dynamic-array approach is more robust and required fewer structural changes to bring in line with the assignment spec.

g. Code Improvements

Starting from GPT-5.6 Luna's baseline, the following changes were made:

1. Fixed the NEXT command's output to match the required multi-line format ("Next email:" / "Sender:" / "Subject:" / "Date:") instead of the slash-delimited single line, which was a correctness failure against the grader's expected output.
2. Verified empty-queue behavior for NEXT and READ produces no output, matching the spec's implicit requirement to handle empty-queue commands gracefully without crashing.
3. Kept and confirmed the dynamic array growth strategy (capacity doubles via `realloc`) so the queue never silently drops emails, unlike a fixed-capacity design.

4. Confirmed heapify-up and heapify-down are implemented iteratively rather than recursively, avoiding unnecessary call-stack overhead.

5. Added a full prolog comment block (program name, description, inputs, outputs, author, creation/revision dates, collaborators) and near-line-by-line inline comments throughout, meeting the assignment's comment rubric, which the GenAI outputs did not have.